Projet : Optimisation Par Colonie De Fourmi D'un Chemin Sur Un Paysage Fractal

Cours CSC_4OS02

Júlia Ellen Dias Leite
Helena Guachalla de Andrade
Mateus Bastos Soares

Mars 2026

Table des matières

1 Introduction **2**

2 Méthodologie **2**
2.1 Environnement et Benchmark 2
2.2 Instrumentalisation 2

3 Résultats Baseline **2**
3.1 Profiling Temporel (gprof) 2
3.2 Décomposition temporelle par itération 3

4 Vectorisation (SoA) **3**
4.1 Approche Structure of Arrays 3
4.2 Profiling de la version vectorisée 3
4.3 Analyse de Performance 4

5 Parallélisation OpenMP **4**
5.1 Stratégie de Parallélisation 4
5.2 Accélération en fonction du nombre de threads 5

6 Parallélisation MPI **5**
6.1 Implémentation hybride MPI + OpenMP 5
6.2 Synchronisation des phéromones 5
6.3 Mesures et granularité calcul/communication 6
6.4 Résultats expérimentaux MPI 6
6.5 Stratégie 2 : Décomposition de Domaine 6

7 Analyse des Goulots d'Étranglement **7**

8 Conclusion **7**

1 Introduction

Ce rapport présente l'optimisation progressive d'un simulateur d'algorithme de colonie de fourmis (ACO) appliqué à la recherche de chemins optimaux sur un terrain fractal. Nous détaillons les étapes de vectorisation et parallélisation du code, en nous appuyant systématiquement sur des profils `gprof` pour guider les optimisations.

2 Méthodologie

2.1 Environnement et Benchmark

Pour garantir une comparaison équitable entre les versions, nous utilisons un scénario fixe :

- Graine pseudo-aléatoire : `2026`
- Taille du `grid` : 512×512 cellules
- Nombre de fourmis : $m = 5000$
- Paramètre de bruit : $\alpha = 0{,}7$
- Coefficient d'évaporation : $\beta = 0{,}999$
- Taux d'exploration : $\varepsilon = 0{,}8$
- Position du nid : $(256, 256)$
- Source de nourriture : $(500, 500)$

Les différents profils `gprof` sont obtenus en compilant avec l'option `PROFILE=yes` et en exécutant le binaire pendant plusieurs milliers d'itérations pour lisser les variations de temps.

2.2 Instrumentalisation

Pour l'analyse de l'optimisation du code, nous mesurons séparément les temps d'exécution par itération de deux phases principales : la première est la fonction `advance_time`, responsable du déplacement et de la mise à jour des positions des fourmis ; et la dernière est la fonction `do_evaporation`, qui assure l'évaporation des phéromones.

La fonction responsable du rendu graphique (`Renderer::display`) est également mesurée, et bien qu'une grande partie du temps de calcul lui soit destinée, la parallélisation ne la prendra pas en compte.

3 Résultats Baseline

3.1 Profiling Temporel (gprof)

Le premier profil `gprof` est obtenu sur la version originale (programmation orientée objet), avec une exécution d'environ 3 591 itérations. Le *flat profile* de cette version donne un temps total de **32,30 s**.

Table 1 – Flat profile gprof — version originale (temps total : 32,30 s)

Fonction	**Temps (s)**	**%**	**Appels**	**ms/appel**
`ant::advance`	15,79	48,89	17 955 000	≈ 0,00088
`Renderer::display`	11,33	35,08	3 591	3,16
`advance_time`	4,77	14,77	3 591	1,33
`fractal_land::compute_subgrid`	0,02	0,06	87 376	0,00
Autre (init, main, allocations, ...)	0,39	1,20	–	–
Total	**32,30**	**100 %**		

Le nombre total d'appels à `ant::advance` est cohérent avec une moyenne de 5 000 appels par itération (17,955M appels / 3,591 itérations).

3.2 Décomposition temporelle par itération

En normalisant par le nombre d'itérations, on obtient un temps moyen d'environ 9,0 ms/itération (32,30 s / 3 591 itérations). La contribution des principales phases s'est donnée principalement par l'avancement des fourmis (`ant::advance`) : 48,9 % — 4,4 ms/itération. Ensuite, par le rendu graphique (`Renderer::display`) : 35,1 % — 3,2 ms/itération. À la fin, par l'évaporation et mise à jour de phéromones (`advance_time`) : 14,8 % — 1,3 ms/itération, tandis que la génération du terrain fractal (`fractal_land`) est négligeable puisqu'elle contribue à moins de 0,2 %.

On constate également que la quasi-totalité du temps d'`advance_time` est passée dans les appels individuels à `ant::advance`, ce qui justifie de cibler en priorité cette fonction pour la vectorisation.

4 Vectorisation (SoA)

4.1 Approche Structure of Arrays

Au lieu de maintenir un objet par fourmi, nous utilisons des vecteurs :

- `pos_x[], pos_y[]` : positions
- `state[]` : états (chargée/non chargée)
- `seed[]` : graines PRNG
- `energy[]` : énergie de mouvement

Cette transformation permet d'améliorer la localité mémoire et facilite l'utilisation de SIMD par le compilateur sur les boucles qui parcourent les fourmis.

4.2 Profiling de la version vectorisée

La version vectorisée (*benchmark*) est profilée avec un temps total de **17,73 s** pour **1 792** itérations, soit **9,90 ms/itération**. Le tableau suivant rassemble les informations du profil.

Table 2 – Flat profile gprof — version vectorisée (temps total : 17,73 s)

Fonction	Temps (s)	%	Appels	ms/appel
`advance_time_vectorized`	11,39	64,24	1 792	6,36
`Renderer::display`	5,99	33,78	1 792	3,34
`fractal_land::compute_subgrid`	0,03	0,17	87 376	0,00
Autre (init, main, allocations, ...)	0,32	1,81	–	–
Total	**17,73**	**100 %**		

On observe que l'appel élémentaire `ant::advance` ne figure plus dans le profil, car sa logique est désormais intégrée dans le noyau vectorisé `advance_time_vectorized` qui traite toutes les fourmis via des tableaux contigus.

4.3 Analyse de Performance

En comparant les temps par itération entre la version orientée objet et la version vectorisée, on obtient :

— Baseline : $\approx 9,0$ ms/itération.

— Version vectorisée : $\approx 9,90$ ms/itération.

Dans cette configuration de benchmark, le temps moyen par itération de la version vectorisée est donc légèrement *plus élevé* (environ 1,1 ms de plus), ce qui correspond à un speedup global $S \approx 0{,}91$ (légère régression). En pratique, cela signifie qu'une partie du temps gagné sur la logique de chaque fourmi est compensée par le coût de gestion des nouveaux tableaux et par le fait que le rendu reste une part importante du temps total.

Le noyau de simulation est désormais regroupé dans `advance_time_vectorized`, qui concentre environ 64 % du temps total, confirmant ainsi la pertinence de le cibler avec des optimisations OpenMP et SIMD explicites.

5 Parallélisation OpenMP

5.1 Stratégie de Parallélisation

Les boucles parallélisées dans le noyau vectorisé sont les boucles externes sur les fourmis et sur les cellules du terrain, en s'appuyant sur `#pragma omp parallel for` :

```
// Boucle sur les fourmis
#pragma omp parallel for
for (int i = 0; i < num_ants; ++i) {
    // mise a jour position, etat...
}

// Boucle sur la grille pour l'evaporation des pheromones
#pragma omp parallel for collapse(2)
for (int x = 0; x < width; ++x) {
    for (int y = 0; y < height; ++y) {
        // evaporation et diffusion locale
    }
}
```

Ces boucles sont naturellement parallélisables car chaque fourmi et chaque cellule sont mises à jour de manière indépendante, sous réserve de gérer correctement les accumulations de phéromones (réductions) et l'accès concurrent à la grille.

5.2 Accélération en fonction du nombre de threads

Pour quantifier l'impact d'OpenMP, nous avons mesuré le temps moyen par itération avec 2, 4 et 8 threads, en comparant au temps de la version vectorisée séquentielle (1 thread). Le temps de référence T_1 est d'environ 9,90 ms/itération.

TABLE 3 – Speedup OpenMP : $S_p = T_1/T_p$

Threads	**1**	**2**	**4**	**8**
Temps (ms/it)	9,90	6,06	4,91	6,39
Speedup S_p	1,00	1,64	2,02	1,55

6 Parallélisation MPI

6.1 Implémentation hybride MPI + OpenMP

La version implemente une stratégie hybride combinant MPI et OpenMP. Chaque processus MPI maintient une copie complète de l'environnement, mais ne simule qu'un sous-ensemble des fourmis. À l'intérieur de chaque processus, la boucle `advance_time` reste parallélisée avec OpenMP, de sorte que le parallélisme est organisé sur deux niveaux : MPI répartit les fourmis entre processus, et OpenMP répartit les fourmis locales entre threads.

Le compteur de nourriture a été adapté au contexte distribué. Pendant chaque itération, chaque processus maintient un compteur local correspondant à la quantité de nourriture récoltée par ses fourmis. En fin d'itération, ces compteurs locaux sont agrégés avec `MPI_Allreduce`, de manière à ce que la valeur affichée par le rendu et enregistrée dans les journaux corresponde à la somme globale sur l'ensemble des processus.

Pour éviter la création de multiples fenêtres graphiques, l'interface SDL est restreinte au rang 0. Seul ce processus initialise SDL, crée la `Window` et le `Renderer`, traite les événements utilisateur et effectue les appels à `Renderer::display`. Les autres rangs continuent à exécuter le calcul (mise à jour des fourmis, évaporation, synchronisation des phéromones), mais sans affichage. Cette organisation permet de conserver un parallélisme efficace sans dupliquer l'interface graphique sur chaque processus MPI.

6.2 Synchronisation des phéromones

La synchronisation des phéromones entre processus est gérée par une fonction dédiée `sync_pheromones`. Pour l'implémenter, nous avons fourni un accès direct au tampon interne de la structure `pheronome`, ce qui permet de sérialiser et d'échanger efficacement la carte de phéromones. À chaque itération, chaque processus envoie sa carte locale au rang 0 via `MPI_Gatherv`. Le rang 0 combine ensuite ces cartes et rediffuse le résultat à tous les processus à l'aide de `MPI_Bcast`, de sorte que tous travaillent à nouveau sur un état global cohérent des phéromones à l'itération suivante.

6.3 Mesures et granularité calcul/communication

Le code de la version hybride a également été instrumenté pour fournir des mesures plus fines. À chaque itération, nous chronométrons séparément le temps passé dans `advance_time`, le temps d'évaporation, le temps de mise à jour des phéromones, le temps de rendu (sur le rang 0), le temps total de l'itération ainsi que le temps dédié aux communications MPI (gather, broadcast, réductions). À partir de ces mesures, nous dérivons des métriques simples de granularité, telles que le rapport calcul/communication et une estimation du volume de données synchronisées par appel, afin d'évaluer dans quelle mesure le coût de communication peut limiter le speedup lorsque le nombre de processus augmente.

6.4 Résultats expérimentaux MPI

Nous avons ensuite évalué la version MPI (stratégie de réplication du domaine). Les profils gprof montrent que le temps total est principalement réparti entre la fonction `advance_time`, la synchronisation des phéromones (`sync_pheromones`) et, lorsque le rendu est activé, `Renderer::display`.

TABLE 4 – Stratégie 1 (réplication) : temps et speedup en fonction du nombre de processus

Processus	**1**	**2**	**4**	**8**
Temps total (s)	–	10,07	15,69	5,80
Temps (ms/it)	–	5,63	8,84	3,29
Speedup S_p	1,0	–	–	–
Commun. (%)	0	≈ 23	42	≈ 30

Les mesures montrent que, pour 4 threads, le coût de synchronisation des phéromones représente déjà plus de 40 % du temps total, ce qui limite fortement le speedup potentiel de la stratégie de réplication du domaine. À 8 threads, le temps total diminue (environ 3,29 ms/itération), mais la part relative de la communication reste significative, et la scalabilité reste inférieure à ce qu'on pourrait attendre d'un noyau largement parallélisable.

6.5 Stratégie 2 : Décomposition de Domaine

— Chaque processus maintient un sous-domaine rectangulaire de la carte.
— Gestion locale des fourmis (migration des fourmis entre sous-domaines).
— Échange de bordures de phéromones entre processus voisins.
— Coût de communication réduit (seulement les halos) mais équilibre de charge potentiellement mauvais si la distribution de fourmis est très inhomogène.

Pseudo-algorithme :

1. Partitionner la carte en sous-domaines ($p = \sqrt{n_\text{procs}}$).
2. Échanger les fourmis migrantes sur les bordures.
3. Mettre à jour la couche de bordure depuis les voisins (phéromones).
4. Faire évoluer les fourmis localement (boucles vectorisées).
5. Mettre à jour phéromones localement puis synchroniser les halos.

7 Analyse des Goulots d'Étranglement

Les profils successifs permettent d'identifier plusieurs goulots d'étranglement structurels :

- **Mémoire** : malgré le passage en SoA, les accès à la grille de phéromones restent non séquentiels et entraînent des défauts de cache, particulièrement pour les trajectoires de fourmis très dispersées.
- **Rendu graphique** : `Renderer::display` représente environ 33 % du temps dans la version vectorisée et plus de 35 % dans la baseline, et n'est pas facilement parallélisable au-delà de quelques optimisations internes (double-buffering, batching).
- **Overhead de fenêtre** : lors de petites exécutions, la création de la fenêtre peut dominer le profil (jusqu'à 60–90 % du temps sur certains runs courts), ce qui impose d'utiliser des runs longs pour des mesures représentatives.
- **Communication MPI** : dans les stratégies distribuées, la synchronisation des phéromones et l'échange de fourmis peuvent limiter le speedup si la fréquence de communication est trop élevée.
- **Équilibrage de charge** : la distribution des fourmis entre processus MPI est statique et uniforme en nombre de fourmis, mais le coût réel par fourmi dépend du terrain et des trajectoires suivies. Il peut donc subsister des déséquilibres en temps de calcul d'un rang à l'autre. Le même phénomène existe au niveau OpenMP, où la répartition par défaut du `parallel for` ne tient pas compte de cette variabilité de charge.

8 Conclusion

Les profils `gprof` montrent que la version orientée objet initiale est dominée par `ant::advance` (environ 49 %) et par le rendu (35 %), ce qui a motivé la restructuration en Structure of Arrays. La version vectorisée regroupe désormais la logique dans `advance_time_vectorized` (environ 64 % du temps) et simplifie l'application d'OpenMP et de SIMD, même si, dans la configuration mesurée, le temps par itération reste légèrement supérieur à la baseline (speedup global $S \approx 0{,}91$).